##

Student Name: Eddie Wenker

Course Code: CSCI 460 -- Fall 2026

Assignment Due Date: 10/25/26

GitHub Link: (https://github.com/Wenker-ed/CSCI-460-Programming-Assignment-I)

##

CSCI 460 Project 1

This project is based on a classification model using bank marketing data. The data is from a Portuguese banking institution that uses direct marketing campaigns. They made phone calls to clients and reported whether they have subscribed a term deposit. The data set is primarily used for classification tasks with sixteen features, one target variable, and 45211 examples. During the cleaning and preprocessing phase of the project, I noticed that there were no missing values in the data set. The only challenge I had faced during this step was encoding ten out of the seventeen variables so they could be used in the Naïve Bayes classification model. After the encoding, I split up the features (x) and the target variable (y), and the data was ready to be used for training and testing.

The next part of the project was splitting x and y into training and testing sets. This allows me to let the model see both training sets while keeping a portion of the data separate for the evaluating the model on unseen observations. In this part of the project, I fit the model ten times with a seventy-percent training split and thirty-percent testing split using the built-in function called train_test_split from the scikit-learn library.

For each of the ten iterations, I used a stratified split which means that both the y training and testing sets will represent clients that did and did not subscribe a term deposit. Since the train_test_split function randomly creates these subsets of data, each iteration gave the model a

new set of training and testing data. The Naïve Bayes classifier was then fit on the training sets, then said fitted model was fed the x testing set to predict 0 or 1 for y. I can then use the predicted y set to test the accuracy of the model compared to the y testing set that I set aside before. In other words, it will show us how well the model could predict whether clients have subscribed a term deposit. The accuracy and F1 scores for each iteration are shown in Table 1 below. The average accuracy score of the ten iterations was 0.826 and the average F1 score was 0.841. The standard deviation for both scores was very small at around ~0.003.

The average accuracy of 0.826 indicates that the model correctly classified approximately 82.6% of clients. The average F1 score of 0.841 shows that the model appears to perform well at identifying positive cases (subscribing a term deposit) while limiting false positives and false negatives. The low standard deviations for both metrics demonstrate that the performance of the model did not depend on a certain random subset of the data. Each split of the data produced performance metrics that were very similar and the model was able to generalize for unseen data.

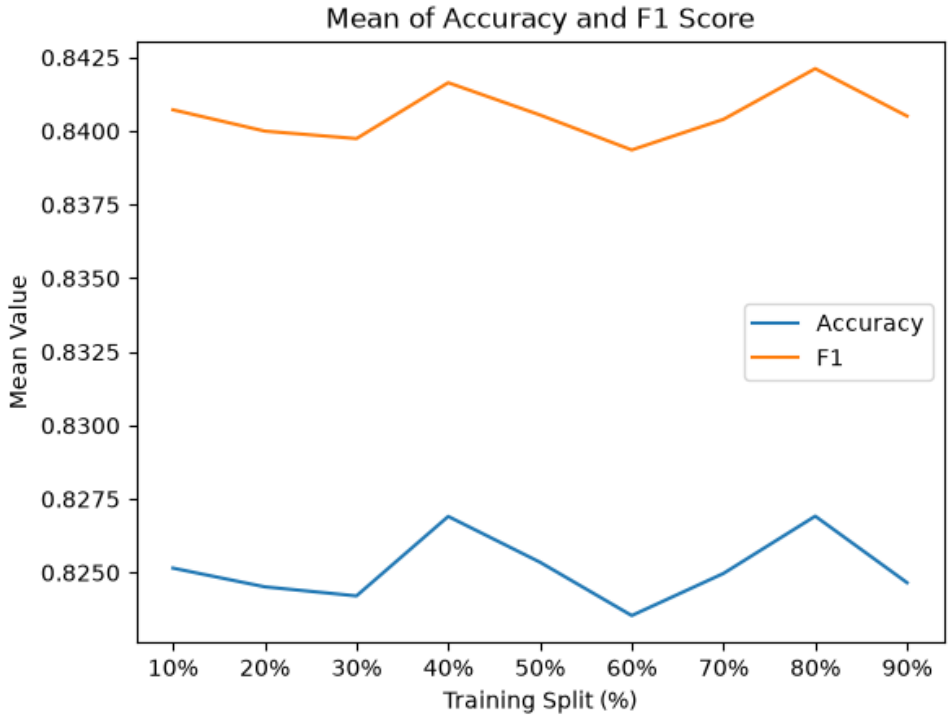
Lastly, I tested out the same ten iteration method for different testing splits to see if there was an ideal amount of data that the model needed to be training on to get the best performance. In Table 2, The mean accuracy and F1 score are similar for each different split. The mean F1 score is consistently ~0.01 higher than the mean accuracy. For some reason, the model does not predict unseen data points better when it sees more training data which is normally the case. As for the standard deviation, it remains roughly the same for each split until the ninety-percent training split where the standard deviation for both accuracy and F1 score was higher than any other split which makes sense because of the higher chance of variability with a much smaller test set.

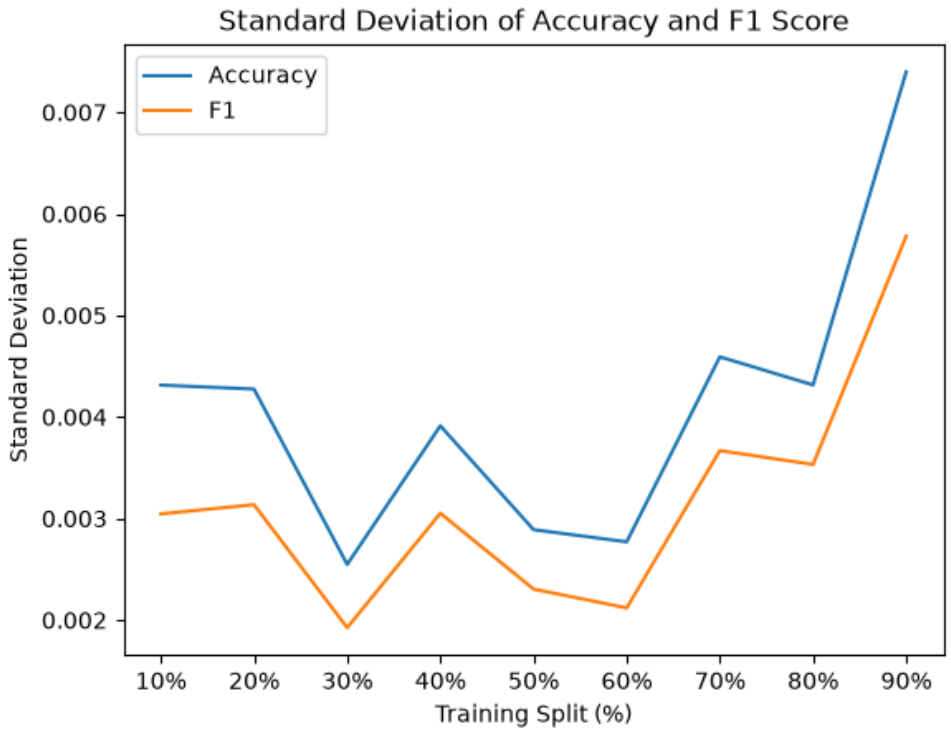
Table 1: 70/30 Split Performance

Iteration	Accuracy	F1 Score
1	0.8307283987024476	0.8448948737787672
2	0.8267472721910941	0.8419108873028164
3	0.831097021527573	0.8449542390195494
4	0.828885284576821	0.8441561220928918
5	0.8249778826304924	0.8411480136579939
6	0.8184163963432616	0.8347825803062032
7	0.8237245650250663	0.8391925534159951
8	0.8325715128280743	0.8458254917972302
9	0.8220289000294898	0.8387011065429943
10	0.8218077263344147	0.8385640259768907
Summary	Average = 0.826 Std. Deviation = 0.004	Average = 0.841 Std. Deviation = 0.003

Table 2: 10-90% Training Split Performance

Training Split	Mean	Standard Deviation
10% training split: Accuracy	0.8251511427869256	0.00431350442249519
10% training split: F1	0.8407229510504732	0.0030448550115565187
20% training split: Accuracy	0.8245154690480797	0.004274193512650406
20% training split: F1	0.8399984003276693	0.0031358839186417086
30% training split: Accuracy	0.8242069009100101	0.0025477619280773983
30% training split: F1	0.8397442895211429	0.0019242109071344667
40% training split: Accuracy	0.8269067718509234	0.003912594088789631
40% training split: F1	0.8416433607362279	0.003051540489120838
50% training split: Accuracy	0.8253516765460496	0.002889832755403783
50% training split: F1	0.8405466613947328	0.002304861097571979
60% training split: Accuracy	0.8235388443461433	0.0027699204654772856
60% training split: F1	0.8393583527976756	0.0021187946434591985
70% training split: Accuracy	0.8249705101739899	0.004591738498707275
70% training split: F1	0.8403985146712485	0.0036681635150725405
80% training split: Accuracy	0.8269158465111136	0.00431465575601091
80% training split: F1	0.8421206750492761	0.0035315956645432 54
90% training split: Accuracy	0.824657231313578	0.007397789158155152
90% training split: F1	0.8405090578893631	0.005779976241573804


Mean of Accuracy and F1 Score
Mean Value
0.8425
0.8400
0.8375
0.8350
0.8325
0.8300
0.8275
0.8250
Accuracy
F1
10%
20%
30%
40%
50%
60%
70%
80%
90%
Training Split (%)


Standard Deviation of Accuracy and F1 Score
Standard Deviation
0.007
0.006
0.005
0.004
0.003
0.002
Accuracy
F1
10%
20%
30%
40%
50%
60%
70%
80%
90%
Training Split (%)